



CSE 4513

Lec – 7

Software Design



All contents are based on textbooks, research articles, online resources, and personal experience.

BAD DESIGN RISK

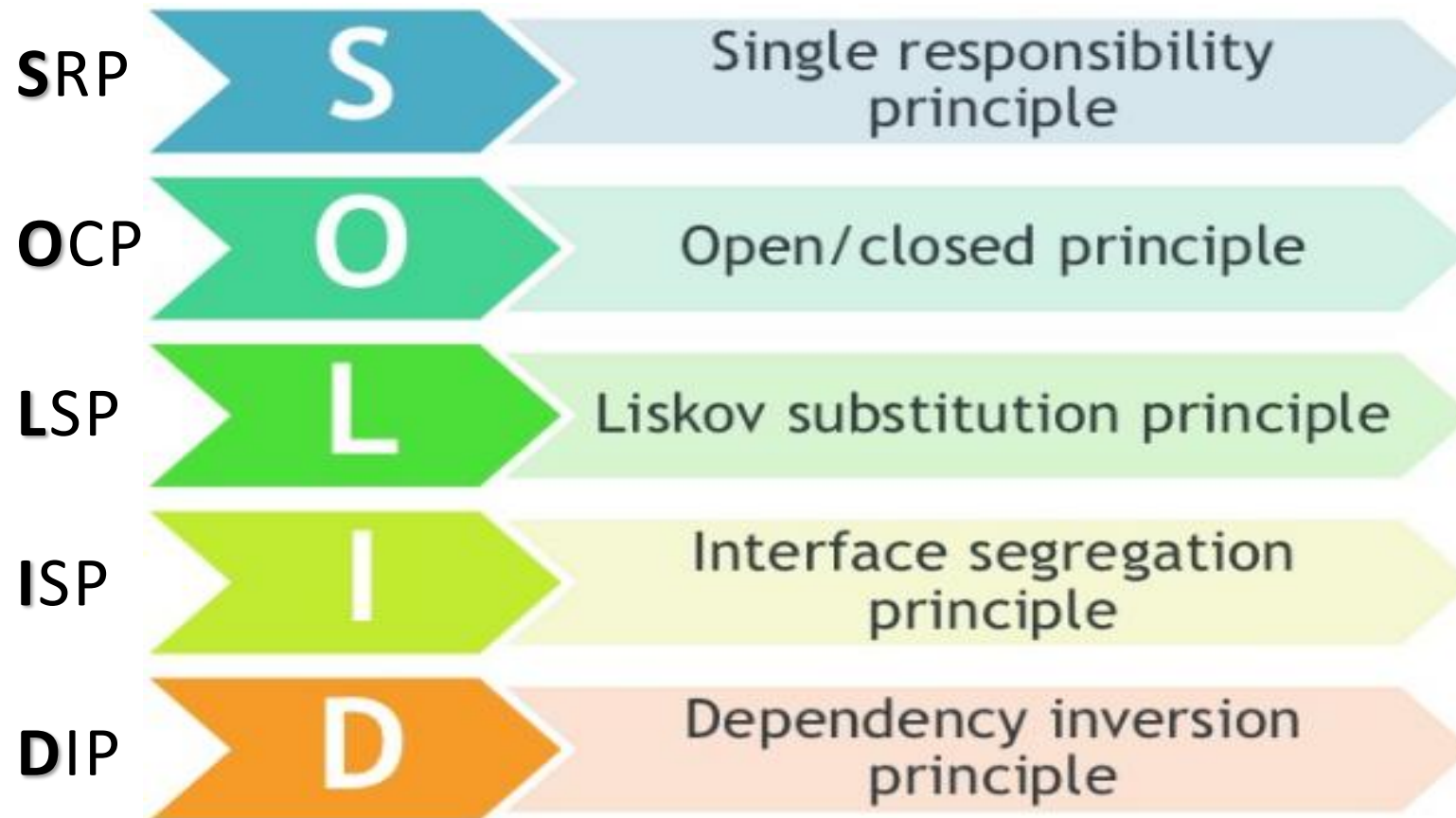


- **103-year-old man asked to bring parents for eye test (Aug 22, 2002)**
 - British pensioner Joseph Dickinson, 103Yrs, had a shock when his local hospital called him in for an eye test and told him to bring his parents.
 - "I must be getting younger, in fact much younger," he told his local paper, the Hartlepool Mail. He was born in 1899.
 - because the hospital computer only read the last two digits it mistook his age as just three years (**103**) old....

"Without good software design, programming is an art of adding bugs to an empty text file" -Louise Srygley

SOLID AGAIN

S.O.L.I.D. PRINCIPLES



SRP: THE SINGLE RESPONSIBILITY PRINCIPLE



Every object should have a single responsibility, and that responsibility should be entirely encapsulated by the class. ~Wikipedia

There should never be more than one reason for a class to change.

~Robert C. “Uncle Bob” Martin



A module should have one, and only one, reason to change.

WHAT IS A RESPONSIBILITY?



- ✓ “a reason to change”
- ✓ A difference in usage scenarios from the client’s perspective
- ✓ More responsibilities == More likelihood of change
- ✓ Having multiple responsibilities within a class couples together these responsibilities

The more classes a change affects, the more likely the change will introduce errors.

COHESION AND COUPLING



Cohesion:

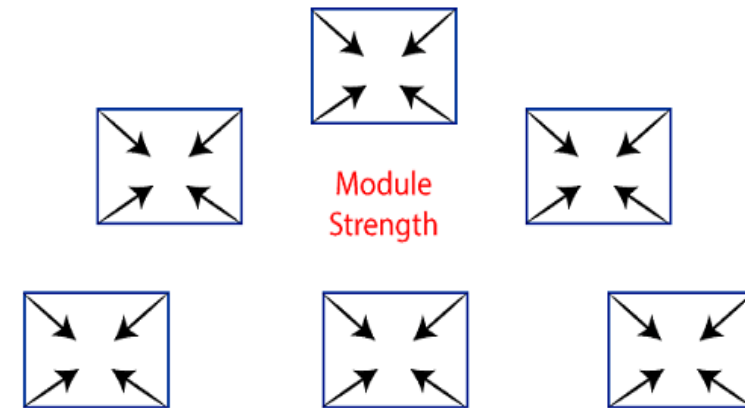
- ✓ refers to how closely related and focused the responsibilities of a module or class are.
- ✓ In a highly cohesive system, each module or class is responsible for a single, well-defined task.

High Cohesion:

- ✓ A module performs one specific task and does it well.
- ✓ High cohesion is desirable because it leads to easier maintainability and readability, and allows for reuse of individual modules or classes.
- ✓ **Example:** In an Address Book application, an Add Module should focus only on adding new addresses. It shouldn't deal with sorting or searching.

Low Cohesion:

- ✓ A module is responsible for many unrelated tasks, leading to a mix of functionality.
- ✓ This makes the system more complex, harder to maintain, and difficult to reuse.
- ✓ **Example:** If an Add Module not only adds addresses but also handles searching and sorting, it has low cohesion. This makes it harder to maintain and extend.



Cohesion= Strength of relations within Modules

COHESION AND COUPLING



Coupling:

- ✓ Coupling refers to the degree of interdependence between different modules or classes.
- ✓ It measures how much one module relies on other modules.

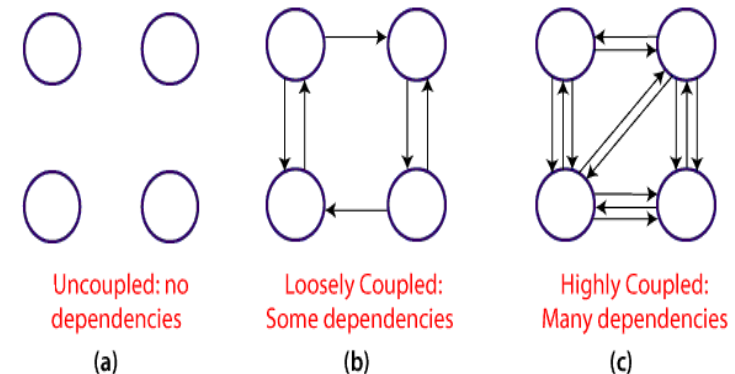
High Coupling:

- ✓ modules depend on each other heavily.
- ✓ A change in one module may force changes in others.
- ✓ High coupling makes the system more rigid and harder to modify or extend because changes in one part propagate through the system.
- ✓ **Example:** In an Address Book app, if the Add Module directly manipulates the storage structure (e.g., file or database) and the storage format changes, the Add Module, along with other modules (Find, Sort, etc.), will need to be modified.

Low Coupling:

- ✓ modules depend on each other minimally.
- ✓ easier to make changes in one module without affecting others
- ✓ **Example:** If the Add Module interacts with a Read/Write Interface, which abstracts storage operations, then any change in the storage structure (file to database, for example) will only require changes to the Read/Write Interface, not the Add Module.

Module Coupling



Strive for low coupling and high cohesion!

Reading assignment

<https://prl.ccs.neu.edu/img/p-tr-1971.pdf>

Carnegie Mellon University
Research Showcase @ CMU

Computer Science Department

School of Computer Science

1971

On the criteria to be used in decomposing systems
into modules

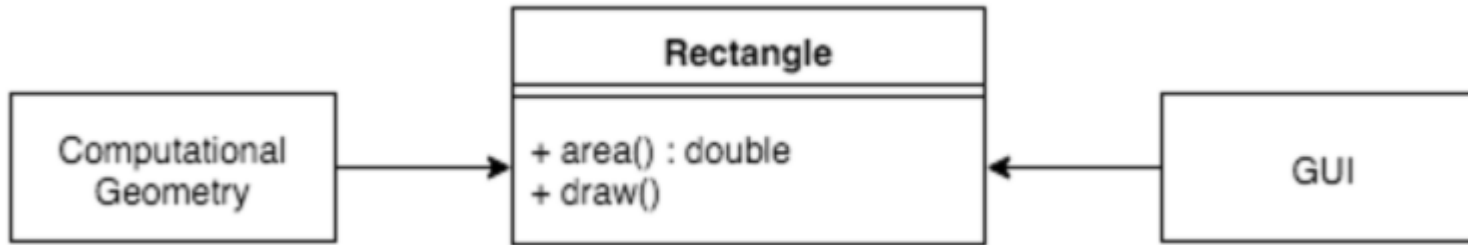
David Lorge Parnas
Carnegie Mellon University

SRP: THE SINGLE RESPONSIBILITY PRINCIPLE

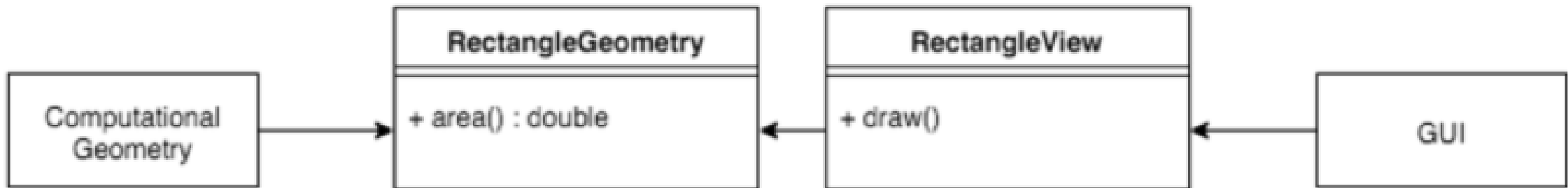


- ✓ Each class should have **one responsibility**, one **single purpose**.
- ✓ This means that a class will do only one job, which leads us to conclude it should have only **one reason to change**.
- ✓ if we have a class that we change a lot, and for different reasons, then this class should be broken down into more classes, each handling a single concern. Surely, if an error occurs, it will be easier to find.

SRP EXAMPLE



- ✓ Here , a rectangle class that has two methods; Area () & Draw ().
 - Area () has the responsibility to return the area of a rectangle
 - Draw () has the responsibility to draw the rectangle itself
- ✓ If there are changes in the GUI, we must modify the Rectangle class.
- ✓ Divide the class in two, so that each class has a unique responsibility.



Now, One will be responsible for calculating and another one for painting.

MORE EXAMPLE



Consider a class for trading application that buys and sells. And we implement a class, named **Transaction**, has two responsibilities: *buying* and *selling*.

```
// Class to handle both Buy and Sell actions
class Transaction {
    // Method to Buy, implemented in Transaction class
    private void Buy(String stock, int quantity, float price){
        // Buy stock functionality implemented here
    }
    // Method to Sell, implemented in Transaction class
    private void Sell(String stock, int quantity, float price){
        // Sell stock functionality implemented here
    }
}
```

If the requirements for either method change it would necessitate a change in the **Transaction** class. In other words, **Transaction** has multiple responsibilities.

MORE EXAMPLE



Refactoring these methods into separate classes would be one approach to applying the Single Responsibility Principle to this application.

```
class Transaction{
    private void Buy(String stock, int quantity, float price){
        Buy.execute(stock, quantity, price);
    }
    private void Sell(String stock, int quantity, float price){
        Sell.execute(stock, quantity, price);
    }
}

class Buy{
    static void execute(String ticker, int quantity, float price){
    }
}

class Sell{
    static void execute(String ticker, int quantity, float price){
    }
}
```

- ✓ Now, Each of those refactored classes is assigned with a single responsibility.
- ✓ The **Transaction** class can still perform two separate tasks but is no longer responsible for their implementation.

OCP: OPEN/CLOSED PRINCIPLE



- The Open / Closed Principle states that software entities (classes, modules, functions, etc.) should be open for **extension**, but closed for **modification**
- In other words, once a module is developed and working, it should be able to accommodate new functionality **without altering its existing codebase**.

A software artifact should be open for extension but closed for modification.

THE OPEN / CLOSED PRINCIPLE



- ✓ Open to Extension
 - New behavior can be added in the future.
- ✓ Closed to Modification
 - Changes to source or binary code are not required.
- ✓ The "closed" part of the rule states that once a module has been developed and tested, the code should only be changed to correct bugs.
- ✓ The "open" part says that you should be able to extend existing code in order to introduce new functionality.

Dr. Bertrand Meyer originated the OCP term in his 1988 book, **Object Oriented Software Construction**

THE OPEN / CLOSED PRINCIPLE



This approach allows **extended** functionality via concrete implementation classes without necessitating changes to **base classes**. In other words, a developer **can add new functionality without changing the existing code** of related functions, classes, and methods!

OCP EXAMPLE



Let's say that we've got a Rectangle class.

```
public class Rectangle
{
    public double Width { get; set; }
    public double Height { get; set; }
}
```

Now our customer, wants us to build an application that can calculate the total area of a collection of rectangles.

```
public class AreaCalculator
{
    public double Area(Rectangle[] shapes)
    {
        double area = 0;
        foreach (var shape in shapes)
        {
            area += shape.Width*shape.Height;
        }

        return area;
    }
}
```

OCP EXAMPLE



We present our solution, the AreaCalculator class to client and he signs us his praise. But he also wonders if we couldn't extend it so that it could calculate the area of not only rectangles but of circles as well.

That complicates things a bit but after some pondering we come up with a solution where we change our Area method to accept a collection of objects instead of the more specific Rectangle type.

```
public double Area(object[] shapes)
{
    double area = 0;
    foreach (var shape in shapes)
    {
        if (shape is Rectangle)
        {
            Rectangle rectangle = (Rectangle) shape;
            area += rectangle.Width*rectangle.Height;
        }
        else
        {
            Circle circle = (Circle)shape;
            area += circle.Radius * circle.Radius * Math.PI;
        }
    }

    return area;
}
```

The solution works and client is happy

Only, a week later he calls us and asks: “extending the AreaCalculator class to also calculate the area of triangles isn’t very hard, is it?”. Of course in this very basic scenario it isn’t but it does require us to modify the code. That is, AreaCalculator isn’t **closed for modification** as we need to change it in order to extend it. Or in other words: it isn’t **open for extension**.

In a real world scenario where the code base is ten, a hundred or a thousand times larger and modifying the class means redeploying it’s assembly/package to five different servers that can be a pretty big problem.

A SOLUTION ABIDES BY THE OPEN/CLOSED PRINCIPLE



One way of solving this puzzle would be to create a base class for both rectangles and circles as well as any other shapes that client can think of which defines an abstract method for calculating it's area.

```
public abstract class Shape
{
    public abstract double Area();
}
```

A SOLUTION ABIDES BY THE OPEN/CLOSED PRINCIPLE



Inheriting from Shape the Rectangle and Circle classes now looks like this:

```
public class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }
    public override double Area()
    {
        return Width*Height;
    }
}

public class Circle : Shape
{
    public double Radius { get; set; }
    public override double Area()
    {
        return Radius*Radius*Math.PI;
    }
}
```

A SOLUTION ABIDES BY THE OPEN/CLOSED PRINCIPLE



As we've moved the responsibility of actually calculating the area away from AreaCalculator's Area method it is now much simpler and robust as it can handle any type of Shape that we throw at it.

```
public double Area(Shape[] shapes)
{
    double area = 0;
    foreach (var shape in shapes)
    {
        area += shape.Area();
    }

    return area;
}
```

In other words we've closed it for modification by opening it up for extension.

MORE EXAMPLE



- ✓ In an e-commerce application, let's say you start by supporting payments via credit cards. As your business grows, you added new payment methods like PayPal.
- ✓ Every time a new payment method is introduced, you need to modify the `PaymentProcessor` class, violating OCP.
- ✓ To achieve OCP, we can create a `PaymentProcessor` interface and use specific implementations for each payment method.

```
class PaymentProcessor {  
    public void processPayment(String paymentMethod) {  
        if (paymentMethod.equals("CreditCard")) {  
            // process credit card payment  
        } else if (paymentMethod.equals("PayPal")) {  
            // process PayPal payment  
        }  
    }  
}
```

MORE EXAMPLE



- ✓ In an e-commerce application, let's say you start by supporting payments via credit cards. As your business grows, you added new payment methods like PayPal.
- ✓ Every time a new payment method is introduced, you need to modify the `PaymentProcessor` class, violating OCP.
- ✓ To achieve OCP, we can create a `PaymentProcessor` interface and use specific implementations for each payment method.
- ✓ Now, to add a new payment method (e.g., GooglePay), you simply implement a new class without modifying the existing code.

```
class GooglePayPayment implements PaymentProcessor {  
    public void processPayment() {  
        // Process GooglePay payment  
        System.out.println("Processing GooglePay payment");  
    }  
}
```

```
// Interface for payment processing  
interface PaymentProcessor {  
    void processPayment();  
}  
  
// Concrete class for credit card payment  
class CreditCardPayment implements PaymentProcessor {  
    public void processPayment() {  
        // Process credit card payment  
        System.out.println("Processing credit card payment");  
    }  
}  
  
// Concrete class for PayPal payment  
class PayPalPayment implements PaymentProcessor {  
    public void processPayment() {  
        // Process PayPal payment  
        System.out.println("Processing PayPal payment");  
    }  
}
```

WHEN DO WE APPLY OCP?



Experience Tells You

- ✓ If you know from your own experience in the problem domain that a particular class of change is likely to recur, you can apply OCP up front in your design

Otherwise – “Fool me once, shame on you; fool me twice, shame on me”

- ✓ Don't apply OCP at first
- ✓ If the module changes once, accept it.
- ✓ If it changes a second time, refactor to achieve OCP

Remember

- ✓ OCP adds complexity to design
- ✓ No design can be closed against all changes

LSP: THE LISKOV SUBSTITUTION PRINCIPLE



- ✓ Child classes should never break the parent class' type definitions.
- ✓ The concept of this principle was introduced by Barbara Liskov in a 1987.
- ✓ a subclass should **extend** the behavior of a parent class without changing its fundamental behavior.



- ✓ If you replace an object of the base class with an object of the derived class, the behavior should remain consistent.
- ✓ The subclass should not introduce behavior that violates the expectations of the base class.
- ✓ The derived class should not break the functionality that the parent class guarantees.

Key Points of LSP:

- 1. Inheritance should not break functionality:** When a class is extended, the derived class should not cause unexpected side effects or break the functionality of the existing code.
- 2. Subclasses should behave like their parent:** Any new subclass should be usable anywhere that the parent class is expected without altering the desired behavior.

SUBSTITUTABILITY



Child classes must not:

- 1) Remove base class behavior
- 2) Violate base class invariants

Substitution Principle: If B is a subtype of A, everywhere the code expects an A, a B can be used instead and the program still satisfies its specification

In general, OOP teaches use of IS-A to describe child classes' relationship to base classes
BUT

LSP suggests that IS-A should be replaced with IS-SUBSTITUTABLE-FOR

LET'S DO AN EXAMPLE!



Let's check the below code:

```
class Transportation
{
    String name;
    String getName() { ... }
    void setName(String n) { ... }
    double speed;
    double getSpeed() { ... }
    void setSpeed(double d) { ... }
    Engine engine;
    Engine getEngine() { ... }
    void setEngine(Engine e) { ... }
    void startEngine() { ... }
}
class Car extends Transportation
{
    @Override
    void startEngine() { ... }
}
```

There is no problem here, right? A car is definitely a **transportation**, and here we can see that it overrides the `startEngine()` method of its superclass.

Let's add another **transportation**:

```
class Bicycle extends Transportation
{
    @Override
    void startEngine() /*problem!*/
}
```

Everything isn't going as planned now!

a bicycle is a transportation , however, it does not have an engine so the method `startEngine()` cannot be implemented.

LET'S DO AN EXAMPLE!



We can refactor our **Transportation** class as follows:

```
class Transportation
{
    String name;
    String getName() { ... }
    void setName(String n) { ... }
    double speed;
    double getSpeed() { ... }
    void setSpeed(double d) { ... }
}
```

we can extend Transportation for non-motorized devices.

```
class TransportWithoutEngine extends Transportation
{
    void startMoving() { ... }
}
```

Now we can extend Transportation for motorized devices.

```
class TransportWithEngine extends Transportation
{
    Engine engine;
    Engine getEngine() { ... }
    void setEngine(Engine e) { ... }
    void startEngine() { ... }
}
```

And Bicycle class is also in compliance with the Liskov Substitution Principle. Now, Car class becomes more specialized, while adhering to the Liskov Substitution Principle.

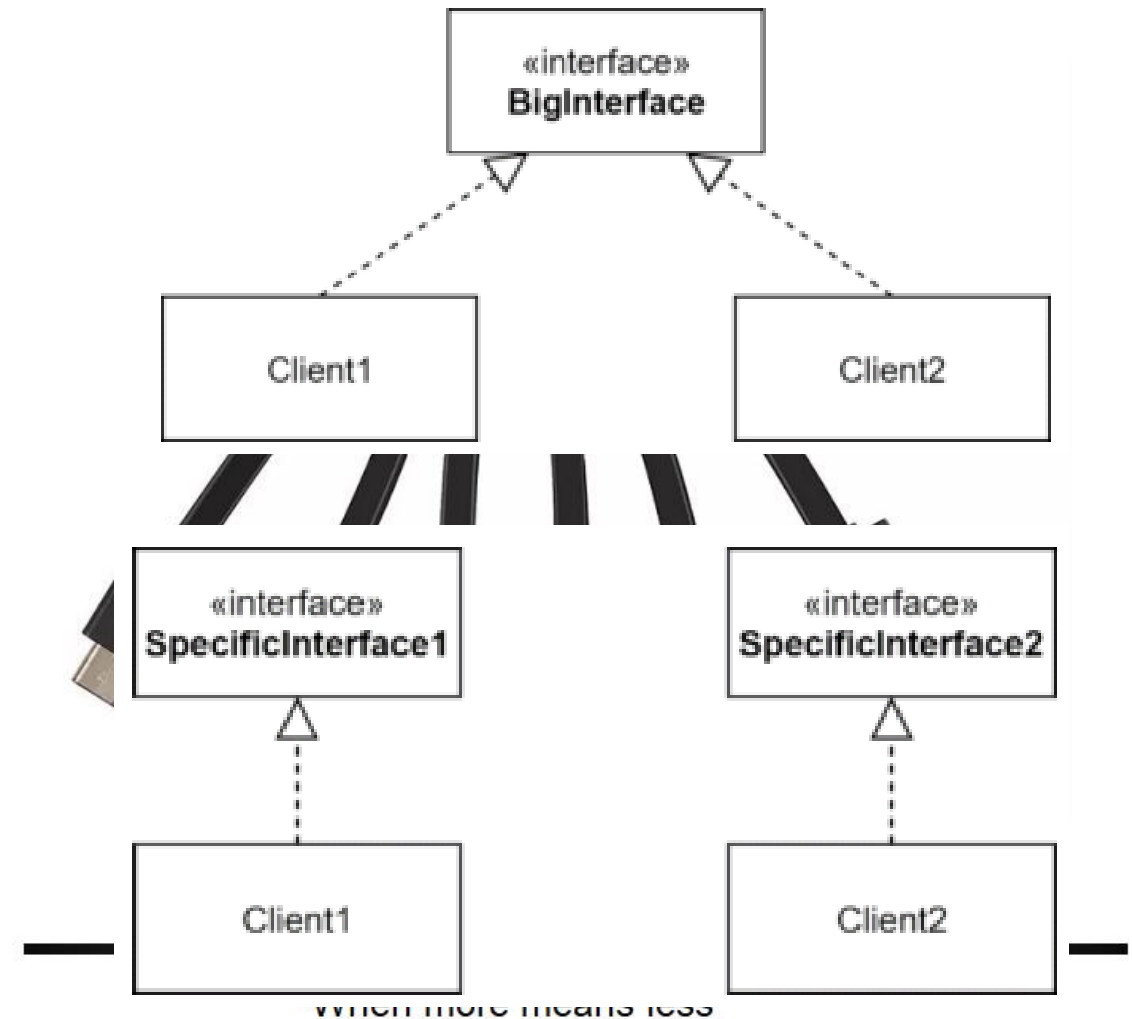
```
class Bicycle extends TransportWithoutEngine
{
    @Override
    void startMoving() { ... }
}
```

This demonstrates the LSP in action because the derived classes (**TransportWithoutEngine** and **TransportWithEngine**) are substitutable for the base class (**Transportation**) without any issues, and the program behaves correctly.

INTERFACE SEGREGATION PRINCIPLE



- ✓ Clients should not be forced to depend on methods they do not use.
- ✓ This means the methods of an interface can be broken up into multiple groups, where each group serves a different client.
- ✓ In other words, one group is there for one client, while the other group is there for another one.
- ✓ is very much related to the Single Responsibility Principle
- ✓ According to the interface segregation principle, you should break down “fat” interfaces into more granular and specific ones.

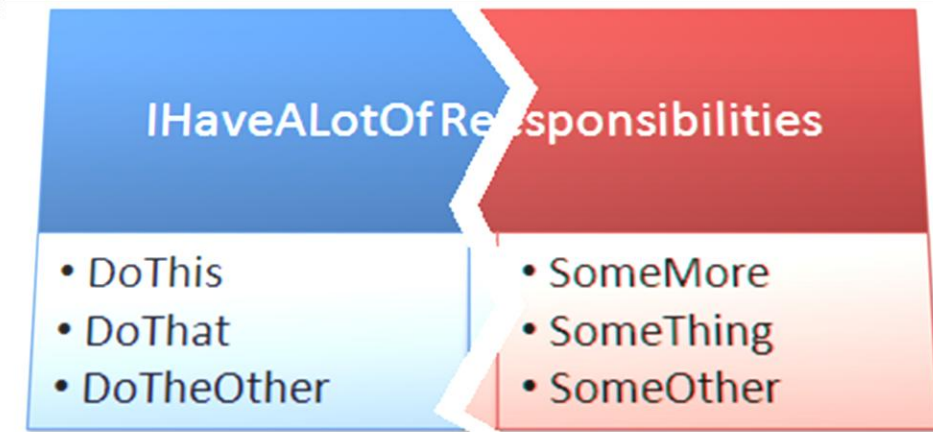


INTERFACE SEGREGATION PRINCIPLE

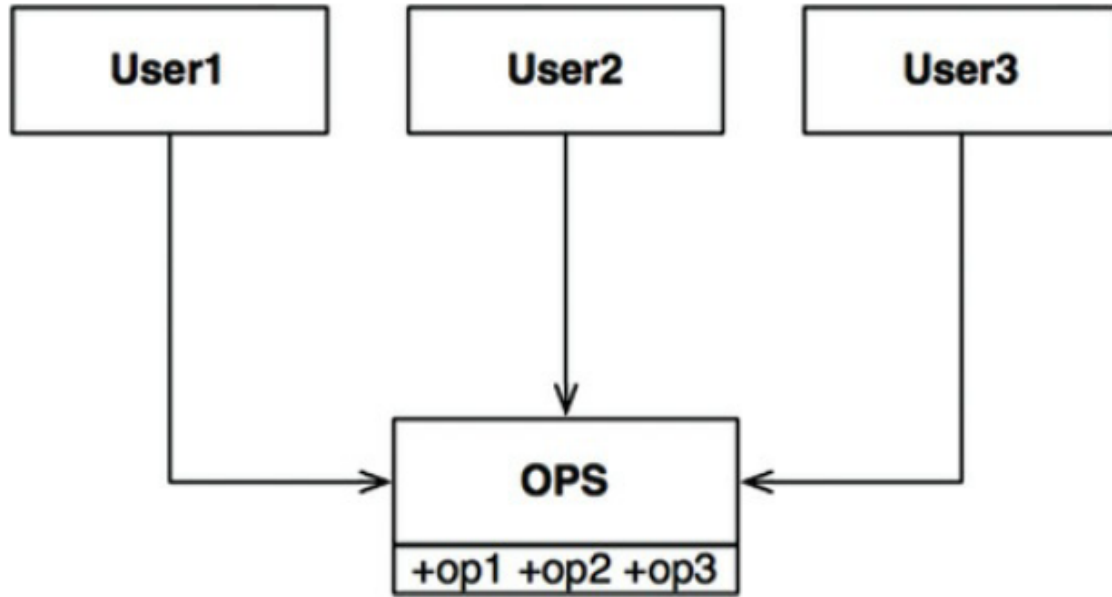


“This principle deals with the disadvantages of ‘fat’ interfaces. Classes that have ‘fat’ interfaces are classes whose interfaces are not cohesive. In other words, the interfaces of the class can be broken up into groups of member functions. Each group serves a different set of clients. Thus some clients use one group of member functions, and other clients use the other groups.”

- Robert Martin

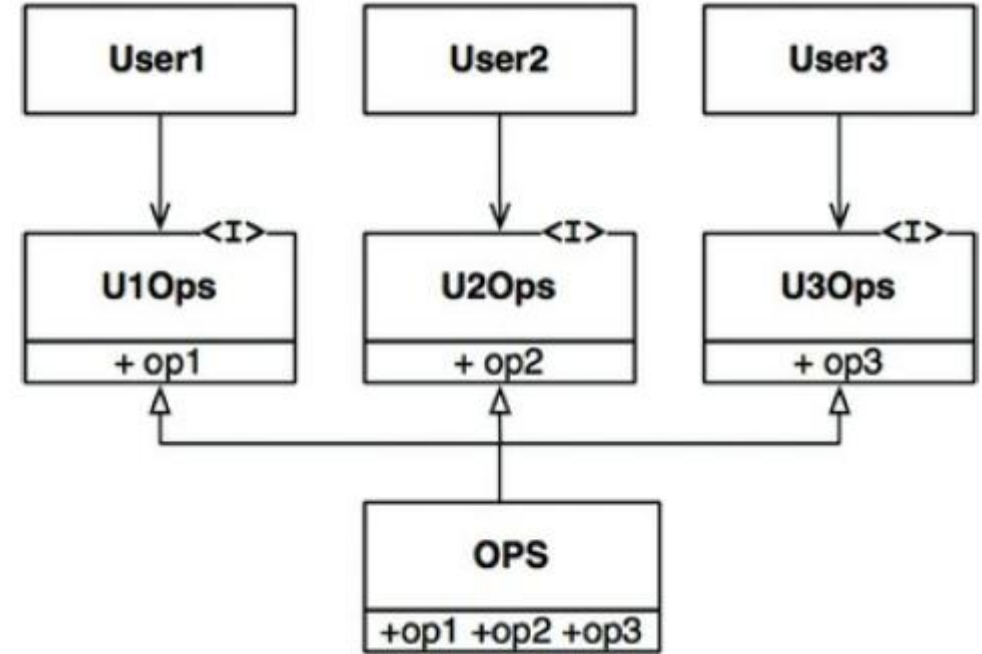


INTERFACE SEGREGATION PRINCIPLE



assume that `User1` uses only `op1`, `User2` uses only `op2`, and `User3` uses only `op3`.

the source code of `User1` will inadvertently depend on `op2` and `op3`, even though it doesn't call them. This dependence means that a change to the source code of `op2` in `OPS` will force `User1` to be recompiled and redeployed, even though nothing that it cared about has actually changed.



Now the source code of `User1` will depend on `U1Ops`, and `op1`, but will not depend on `OPS`. Thus a change to `OPS` that `User1` does not care about will not cause `User1` to be recompiled and redeployed.

WHEN DO WE FIX ISP?



- ✓ Once there is pain
 - If there is no pain, there's no problem to address.

In fat interfaces, there are too many operations, but for most objects, these operations are not used.

DEPENDENCY INVERSION PRINCIPLE



- ✓ **High-level modules should not depend on low-level modules.** Instead, both should depend on **abstractions (like interfaces or abstract classes)**. Abstractions should not depend on details. Details should depend on abstractions.

WHAT IS DEPENDENCY



A dependency is something - another class, or a value - that your class depends on, or requires. For example:

- ✓ If a class requires some setting, it might call

ConfigurationManager.AppSettings["someSetting"].

ConfigurationManager is a dependency. The class has a dependency on *ConfigurationManager*.

- ✓ If you write two classes - let's say *Log* and *SqlLogWriter* - and *Log* creates or requires an instance of *SqlLogWriter*, then *Log* depends on *SqlLogWriter*.
SqlLogWriter is a dependency.

DEPENDENCIES....



- ✓ Framework
- ✓ Third Party Libraries
- ✓ Database
- ✓ File System
- ✓ Email
- ✓ Web Services
- ✓ System Resources (Clock)
- ✓ Configuration
- ✓ The new Keyword
- ✓ Static methods
- ✓ Thread.Sleep
- ✓ Random

HIGH LEVEL MODULES AND LOW LEVEL MODULES



The principle says that “both should depend on abstractions.” means that when it comes to applying the principle, the difference between high level and low level doesn’t matter.

WHAT ARE ABSTRACTIONS?



- ✓ Typically, **interfaces** or **abstract classes** serve as abstractions, as they define methods and properties without implementing the actual logic.
- ✓ **Purpose:**
 - ✓ Abstractions define "what" something does without specifying "how" it does it.
- ✓ **Example:**
 - ✓ An interface `PaymentProcessor` could represent an abstraction for any kind of payment processor. It might define a method `processPayment`, which different payment methods can implement.

```
// Abstraction: PaymentProcessor interface defines a common behavior  
public interface PaymentProcessor {  
    void processPayment(double amount);  
}
```

PRINCIPLES OF COMPONENT DESIGN



- ✓ If SOLID principles tell us how to arrange the bricks into walls, then the component principles tell us how to arrange the rooms into buildings.
- ✓ Large software systems, like large buildings, are built out of smaller components.
- ✓ Components are smallest entities that can be **deployed** as part of a system.
- ✓ In Java, they are jar files. In Ruby, they are gem files. In .Net, they are DLLs.
- ✓ well-designed components always retain the ability to be **independently deployable** and, therefore, **independently developable**.



PRINCIPLES OF COMPONENT DESIGN



They are divided into two categories:

- ✓ **Principles of Component Cohesion** which focus on the granularity of components and help the developer partition classes into components
- ✓ **Principles of Component Coupling** which focus on the stability and relationships between the components.



COMPONENT COHESION



Which classes belong in which components?

Three principles of component cohesion:

- **REP:** The Reuse/Release Equivalence Principle
- **CCP:** The Common Closure Principle
- **CRP:** The Common Reuse Principle



THE REUSE/RELEASE EQUIVALENCE PRINCIPLE



- ✓ Classes and modules that are grouped together into a component should be releasable together.
- ✓ is closely related to versioning and release management in software development.
- ✓ It suggests that software components should be **organized and released** based on their **reuse potential**, meaning that components that are likely to be **reused together** should be **packaged and versioned** together.
- ✓ Reuse/Release Equivalence Principle emphasizes that the organization of components and their versioning should align with their patterns of reuse.

What is reuse?

THE REUSE/RELEASE EQUIVALENCE PRINCIPLE



Example:

If you are building a logging library, you would want to bundle all related classes (e.g., `Logger`, `LogFormatter`, `LogWriter`) into a single, versioned library package.

This allows developers to add it to their projects as one reusable module, ensuring that they can get updates or bug fixes by upgrading to a new version without dealing with individual classes.

By adhering to the REP, software components become more manageable, maintainable, and reliable, improving the overall quality of the system and making reusability easier to achieve.

THE COMMON CLOSURE PRINCIPLE



Gather into components those classes that change for the same reasons and at the same times.

Separate into different components those classes that change at different times and for different reasons

Just as the SRP says that a class should not contain multiples reasons to change, so the Common Closure Principle (CCP) says that a **component should not have multiple reasons to change**.

THE CLASSES IN A PACKAGE SHOULD BE CLOSED TOGETHER AGAINST THE SAME KINDS OF CHANGES. A CHANGE THAT AFFECTS A PACKAGE AFFECTS ALL THE CLASSES IN THAT PACKAGE.

THE COMMON CLOSURE PRINCIPLE



Example:

Imagine you're building an invoicing system. You might have classes like: `Invoice`, `InvoiceCalculator`, `InvoicePrinter`

- ✓ These classes all relate to the invoicing functionality.
- ✓ If there's a change in the invoicing requirements (such as changes in tax calculations or formatting), it's likely to impact all three classes.
- ✓ According to CCP, these classes should be grouped in the same module or package, such as `invoice`, since they are likely to change together.

The CCP is about grouping classes together that are likely to **change for the same reason**. This makes the code more organized, easier to maintain, and less likely to break from changes.

THE COMMON REUSE PRINCIPLE



THE CLASSES IN A PACKAGE ARE REUSED TOGETHER. IF YOU REUSE ONE OF THE CLASSES IN A PACKAGE, YOU REUSE THEM ALL.

Don't force users of a component to depend on things they don't need.

CRP is another principle that helps us to decide which classes and modules should be placed into a component. It states that classes and modules that tend to be reused together belong in the same component.

CRP says that classes that are not tightly bound to each other should not be in the same component.

COMPONENT COUPLING



Relationships between components

Three principles of component Coupling:

- The Acyclic Dependencies Principle
- The Stable Dependencies Principle
- The Stable Abstractions Principle

THE STABLE DEPENDENCIES PRINCIPLE



Depend in the direction of stability.

What is stability.

Stability is related to the amount of work required to make a change.

"**Stable**" roughly means "**hard to change**", whereas "**instable**" means "**easy to change**"

A component with lots of incoming dependencies is very stable because it requires a great deal of work to reconcile any changes with all the dependent components.

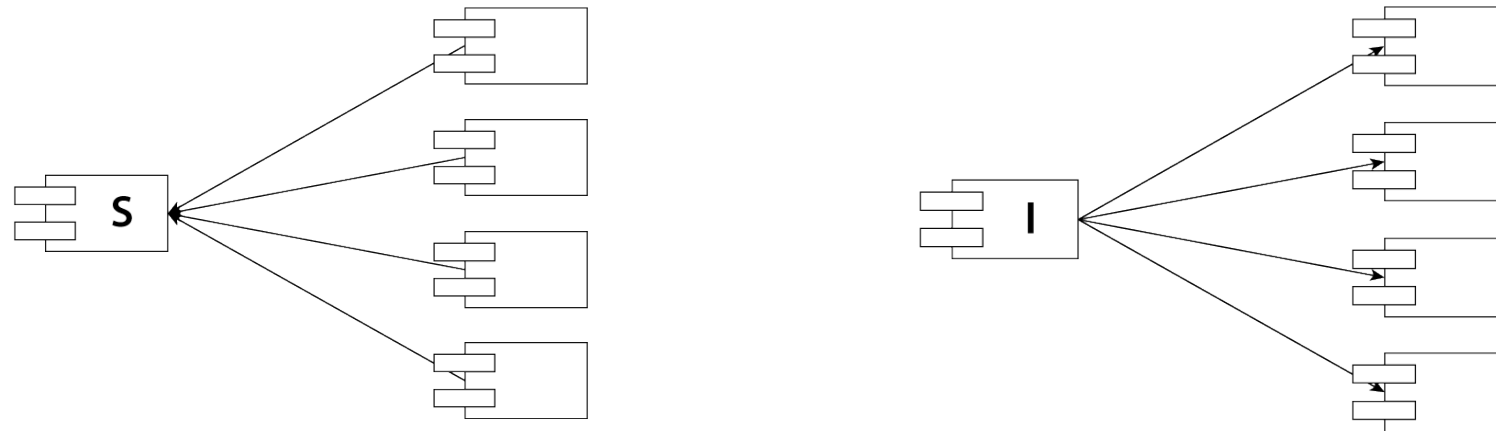


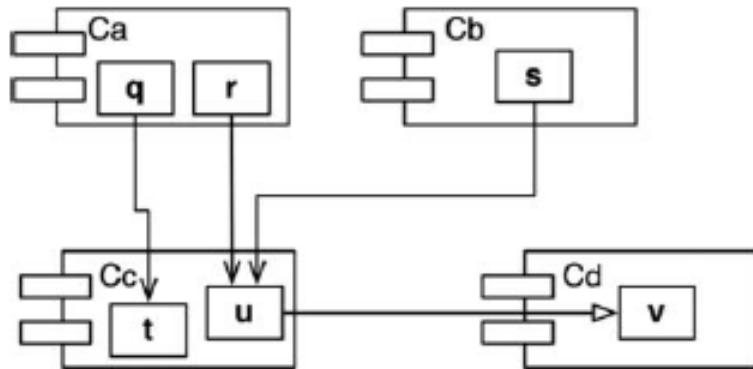
Figure shows *S*, which is a stable component. Four components depend on *S*, so it has four good reasons not to change. We say that *S* is *responsible* to those four components. Conversely, *I* depends on nothing, so it has no external influence to make it change. We say it is *instable*.

CALCULATING COMPONENTS STABILITY



Fan-in: Incoming dependencies. This metric identifies the number of classes outside this component that depend on classes within the component.

- *Fan-out*: Outgoing dependencies. This metric identifies the number of classes inside this component that depend on classes outside the component.
- *I*: Instability: $I = \text{Fan-out} / (\text{Fan-in} + \text{Fan-out})$. This metric has the range [0, 1]. 0 indicates a maximally stable component. 1 indicates a maximally unstable component.



Let's say we want to calculate the stability of the component C_c . We find that there are three classes outside C_c that depend on classes in C_c . Thus, $\text{Fan-in} = 3$. Moreover, there is one class outside C_c that classes in C_c depend on. Thus, $\text{Fan-out} = 1$ and $I = 1/4$.

Not All Components Should Be Stable

THE STABLE ABSTRACTIONS PRINCIPLE



The more stable a component is, the more abstract it should be

The Stable Abstractions Principle (SAP) sets up a relationship between stability and abstractness.

if a component is to be stable, it should consist of interfaces and abstract classes so that it can be extended.

How abstract a component is simply the ratio of interfaces and abstract classes in a component to the total number of classes in the component.

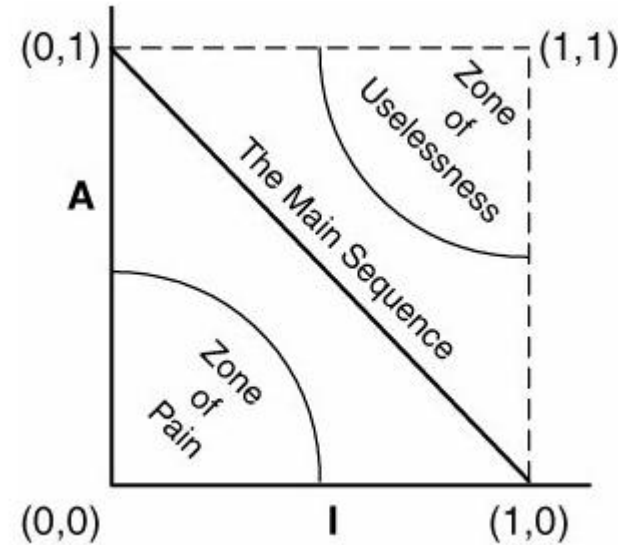
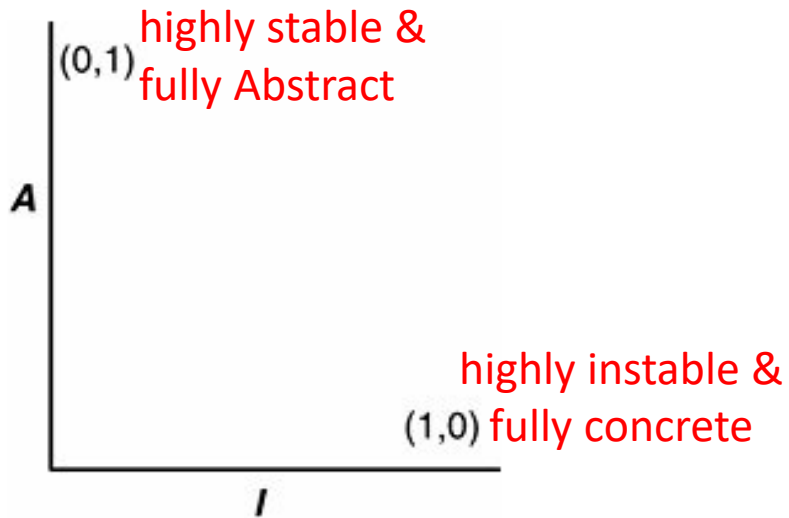
N_c : The number of classes in the component.

N_a : The number of abstract classes and interfaces in the component.

A : Abstractness. $A = N_a / N_c$.

The A metric ranges from 0 to 1. A value of 0 implies that the component has no abstract classes at all. A value of 1 implies that the component contains nothing but abstract classes.

RELATIONSHIP BETWEEN STABILITY (I) AND ABSTRACTNESS (A)



Consider a component in the area near (0,0). This component is highly stable and concrete. Such a component is not desirable, because it is rigid. It cannot be extended, because it is not abstract. And it is very difficult to change, because of its stability.